BioHackrXiv

SWAT4HCLS26 report: OpenMRS as a reference implementation platform for Electronic Health Records and Clinical Decision Support

Andra Waagmeester [1], **Stephanie "Ace" Medlock** [1,2], **Eric Herman**[2,3], **Claude Nanjo**[4], **Chang Sun** [5], **and Ronald Cornet** [1]

1 Department of Medical Informatics, AmsterdamUMC, Location AMC Amsterdam, the Netherlands **2** Stichting OpenElectronicsLab, The Netherlands **3** Commons Caretakers B.V., Amsterdam, The Netherlands **4** University of Utah **5** DACS - Data Department of Advanced Computing Sciences, Faculty of Science and Engineering, Maastricht, the Netherlands

BioHackathon series:
Biohackathon SWAT4HCLS 2026
AmsterdamUMC
OpenMRS

Submitted: 19 Jun 2026

Published by BioHackrXiv.org

Introduction

Electronic health records (EHRs) have become the central piece of software in modern health care (Shen et al., 2025). Most patient care is documented and managed in the EHR, and functionality to support clinical decisions is therefore best integrated into the EHR as well (Kawamoto et al., 2005). The shareability of clinical decision support systems (CDSSs) is a longstanding challenge, partly due to the limited interoperability of EHRs (Shen et al., 2025; Sittig et al., 2008). This makes it difficult for researchers to show how a CDSS, and especially a novel CDSS, could be integrated into an EHR. Innovations in CDSSs can be disseminated by publishing a specification, but a *reference implementation* of the CDSS goes further: it helps potential users imagine how the CDSS would fit into their workflow, and it helps technical implementers understand what is required to fully realise the specification as an EHR-integrated CDSS.

Open-source EHRs make it possible to create such reference implementations. The open-source licensing model gives users the freedom to adapt the software to their needs (Free Software Foundation, n.d.). This ranges from using, or adding, functionality that implements existing interoperability standards, through to adding new functionality that demonstrates novel means of deeply integrating new kinds of support. A working, inspectable system removes the guesswork that a written specification alone leaves open.

Open-source EHRs also support the education of medical informatics professionals. EHRs are central to modern health care, yet it is common for medical informatics students to graduate without ever having used one. The open-source licensing model gives users the "freedom to inspect" (Free Software Foundation, n.d.), meaning that students can "look under the hood" of an EHR and gain hands-on experience with the secondary uses of EHR data, including uses that require interoperability.

Because of the authors' previous experience with it (Medlock et al., 2023), OpenMRS was chosen as the target system for our Biohackathon table. OpenMRS is an open-source EHR with an active developer community, a modular architecture, and a modern front end (the "O3" reference application). The broad question for the OpenMRS table was: **What steps are needed to use OpenMRS as a resource for reference implementations?**

This report is an *intermediate* one. Rather than presenting a finished system, it documents where we started, what we tried at the Biohackathon, how far we got at each step, and what remains to be done. The aim is to give the next group of contributors, whether researchers building a reference implementation or teachers building a course, a realistic, plain-language starting point.

A motivating use case: genomic clinical decision support

To make the goal concrete, it helps to have a real CDSS in mind that one might want to integrate with an EHR. One of us (Nanjo) is working toward an open-source platform for personalised medicine, with an initial focus on pharmacogenomics. This use case illustrates *why* an EHR reference implementation is needed, and what such an implementation has to support.

The promise of personalised medicine is that a patient's genomic information can be used to tailor their care, for example by choosing a drug dose that matches how that individual metabolises the drug. In practice, the use of genomic knowledge in daily clinical practice is still limited. The knowledge is growing quickly, most clinicians are not equipped to interpret it at the point of care, and, apart from a few well-established cases, there is no software that brings the relevant guidance into the clinical workflow at the moment a decision is being made. Without dedicated platforms and APIs that deliver this knowledge inside the EHR, the promise of personalised medicine is unlikely to be realised.

Several concrete examples show what such a platform would do:

- **Thiopurine dosing (pharmacogenomics).** Thiopurines (e.g. azathioprine, 6-mercaptopurine) are used in leukaemia, inflammatory bowel disease, and transplantation. A patient's *TPMT* and *NUDT15* genotype predicts the risk of severe, potentially life-threatening myelosuppression, and CPIC guidelines translate that genotype into concrete dose adjustments. A useful CDSS would surface that adjusted dose when the drug is prescribed, not merely a textual warning.
- **Warfarin dosing.** A *CYP2C9* and *VKORC1* genotype, combined with patient factors such as age and weight, can be used to compute an individualised starting dose rather than the standard one.
- **Familial hypercholesterolemia and hereditary cancer risk.** Identifying a pathogenic variant (e.g. in *LDLR*, or in *BRCA1*/*BRCA2*) turns a vaguely "high-risk" patient into a clearly identified one, justifying earlier and more aggressive prevention and enabling cascade testing of relatives.
- **Rare disease screening.** Tools such as PubCaseFinder can suggest candidate rare diseases from a patient's problem list; integrated into a CDSS, the patient's encounter data could be sent to such a service and the results presented back to the clinician.

A recurring lesson from these examples is architectural: the EHR, the CDSS, and the genomic knowledge base should be **decoupled** from one another. The genomic knowledge base may be used in contexts that have nothing to do with a particular EHR; the CDSS reasons over a clinical model that need not be identical to the EHR's; and the EHR is the system of record that the clinician actually works in. In practice, the CDSS connects to the EHR through interoperability standards such as CDS Hooks and SMART-on-FHIR, and to the knowledge base through a separate (and currently still under-specified) API.

This is exactly the kind of integration that is hard to demonstrate with a specification alone, and exactly why an *open-source EHR that can be used as a reference implementation platform* is valuable. The rest of this report is about building that platform, one capability at a time.

Approach: four levels of using OpenMRS as a reference implementation platform

We divided the broad question into four levels of increasing difficulty. Each level unlocks a different kind of demonstration, and each builds on the skills of the previous one.

Level 1: Use OpenMRS as a data source through its existing APIs. A copy of the OpenMRS reference application (OpenMRS community, n.d.-b) was deployed with demo data

and made available during the Biohackathon. The goal at this level is to see whether a demo of some desired functionality can be built *purely* by calling the APIs that the reference application already exposes. (Note: the "OpenMRS reference application" should not be confused with the reference implementation we ultimately want to build. The OpenMRS reference application shows how OpenMRS itself can be deployed; our goal is to build a reference implementation of *novel CDSS functionality integrated with* OpenMRS.)

Level 2: Run OpenMRS locally and access its database directly. The OpenMRS community provides instructions for installing OpenMRS on a local machine (OpenMRS community, n.d.-d). The goal at this level is to follow those instructions, write more detailed ones where needed, and then build a demo that queries the OpenMRS database directly. This supports applications that need EHR data but do not need to interact with the user through the EHR interface, for example building a research cohort.

Level 3: Add a module to OpenMRS. This is the first step toward modifying OpenMRS itself. The OpenMRS community provides "Getting Started as a Developer" instructions (OpenMRS community, n.d.-a). The goal at this level is to follow those instructions (improving them where needed) in order to add a simple module, for example "add a clickable URL to the patient summary screen." This is the mechanism by which new functionality is embedded in OpenMRS, and it is the path a CDSS like the genomic-CDS use case would take if OpenMRS does not already support the data entry it needs.

Level 4: Build OpenMRS from source. To make *deep* changes to OpenMRS, rather than just adding something alongside it, new functionality may have to be added to existing OpenMRS modules or to the core. The goal at this level is to produce instructions for building a minimum viable installation of OpenMRS from source, preferably in an executable form (for example, a script or Makefile) so that the process can be repeated and adapted by others.

An additional, cross-cutting goal of the table was to produce a "roadmap" for EHR education: what medical informatics students should learn about an EHR, and how an open-source EHR can be leveraged to teach it. We return to this in the section on education below.

Results: what we achieved at the Biohackathon

This section reports, level by level, how far we got and what is still blocking progress. Levels 1 and 2 are working; levels 3 and 4 have a reproducible setup that does not yet fully work, with the remaining blockers documented so the next contributor can pick them up.

Level 1: OpenMRS APIs

We deployed the OpenMRS reference application with demo data and confirmed that its existing REST and FHIR APIs can be used as a data source for demos. One practical finding is that the version of FHIR implemented in the reference application is out of date. Contributing to the maintenance of the OpenMRS FHIR module (OpenMRS community, n.d.-c) would therefore be a worthwhile long-term goal, both for this project and for the wider community, since up-to-date FHIR support is what makes standards-based CDSS integration (CDS Hooks, SMART-on-FHIR) possible.

Level 2: Local installation and direct database access

We were able to install and run the reference application locally on both Linux and Windows, and to reach the point where the underlying database can be queried. The Windows path required a long wait for first-time data loading (see below). The reproducible setup we used is captured in *Reproducing our setup* at the end of this section so that others can get to a running, queryable instance quickly.

BioHackrXiv

Level 3: Adding a module (OpenMRS SDK)

We created a sandbox that provisions a virtual machine containing an instance of the OpenMRS SDK (OpenMRS Education, n.d.), the toolchain used to scaffold and run modules. The same VM-based pattern as Level 4 is used: one script creates the VM and runs the build inside it, and a second script lets you SSH in to look around.

This does **not** work end-to-end yet. The main blocker is that the OpenMRS SDK setup is interactive, so the process cannot yet be fully automated. The immediate piece of future work is to drive the SDK non-interactively so that "scaffold and run a minimal module" becomes a single repeatable command. Adding the canonical "clickable URL on the patient summary" module on top of that automated SDK is the natural next milestone.

Level 4: Building OpenMRS from source

We created a sandbox that builds OpenMRS core from source inside a fresh virtual machine (Herman, n.d.). The build runs and reports the git hash of the commit it built, which is an encouraging first step toward a repeatable, reference-quality build process for researchers and students.

It is also not yet complete. The known blockers at the time of writing are:

- the build must check out OpenMRS core `2.8.5`; building from `master` fails with a Hibernate error ("Unable to determine Dialect without JDBC metadata");
- the legacy UI (`legacyui`) is not yet working;
- property/message changes are not yet visible in the running instance.

The value already delivered is the *executable* setup itself: `build-openmrs.sh` captures the actual build process, and the surrounding scripts (`create-vm-and-run-build-openmrs`, `ssh-to-vm`, `re-launch-vm`) make it repeatable on a clean Debian VM. Turning this into a reliable, documented "build OpenMRS from source" recipe is the main Level 4 future-work item.

Reproducing our setup (Levels 1–2)

The following commands reproduce the running reference application we used.

Linux

```
apt update
apt install docker-compose-v2
git clone https://github.com/openmrs/openmrs-distro-referenceapplication.git
cd openmrs-distro-referenceapplication
git tag --sort=-v:refname
TAG=3.5.0 docker compose -f docker-compose.yml up -d
```

Replace the tag with the latest one (or whichever you choose from the list produced by the previous command). After a few minutes, the installation UI is available at `http://localhost:8080/openmrs/`. After roughly 15–45 minutes (depending on the machine), the O3 UI is available at `http://localhost:8080/openmrs/spa/home`. The default login is username `admin`, password `Admin123`.

Windows

```
gh repo clone openmrs/openmrs-distro-referenceapplication
cd openmrs-distro-referenceapplication
docker compose up        # or: docker compose up -d
```

On first run we saw an empty screen / HTTP 504 responses for about 30 minutes while the initial data loaded. When the backend appeared stuck, restarting it helped:

```
docker compose restart backend
```

After waiting roughly 17 more minutes for the backend to finish loading, the O3 UI was reachable at `http://localhost/openmrs/spa` and we could log in with `admin` / `Admin123`.

Open-source EHR in medical informatics education

EHRs are central to modern health care, but they are largely absent from medical informatics education. Students learn about terminologies, standards, relational databases, and decision support in the abstract, yet rarely touch a real EHR, the system in which all of those ideas come together, until they are on the job. An open-source EHR can close that gap: because students are free to install, inspect, and modify it, a single system can host a sequence of learning objectives, from "what is an EHR?" to "build a CDSS integration."

Figure 1 sketches a long-term vision for using an open-source EHR as a teaching and research platform. The figure should be read as a map: a small set of **components** in the centre, and a set of **use cases** (learning objectives, labelled A–M) positioned on the components that exercise them.

At the centre is the **OpenMRS relational database**, with the **OpenMRS (O3) user interface** on top of it. Around them sit the supporting components that turn OpenMRS into a full teaching environment:

- **Synthea**, to generate synthetic patient populations so that the system can be filled with realistic but non-identifiable data;
- a **(HAPI) FHIR server**, exchanging HL7 FHIR patient data with the EHR;
- a **SNOMED terminology server / browser (Snowstorm)**, providing terminology and terminology services;
- **Ontop**, exposing the relational data as a queryable knowledge graph (SPARQL) for cohort selection via an ontology;
- **REDCap**, to model and run a (simulated) clinical trial.

The lettered use cases describe what a student does with these components, and they map onto a curriculum:

- **A. Learning what an EHR is** and **B. human–computer interaction**: using the O3 interface itself;
- **C. Understanding relational databases**: querying the OpenMRS database directly;
- **D. Using terminologies** and **E. using terminology services**: via the SNOMED terminology server;
- **F. Using REST services / FHIR**: exchanging patient data over HL7 FHIR;
- **G. Teaching patients about their data** and **H. allowing data donation by patients**: patient-facing uses of the EHR;
- **I. Integrating decision support**: the CDSS integration that motivates this report;
- **J. OpenMRS development**: the ladder of Levels 1–4 above (via FHIR, via database querying, via the OpenMRS SDK, and building from source);
- **K. Using OpenMRS as a reference implementation platform**: the overarching goal of the table;
- **L. Creating cohorts from patient data using an ontology**: via Ontop and a terminology server;
- **M. Simulating a clinical trial**: via REDCap.

For this vision to be usable in a classroom, two practical **requirements** stand out. First, installation has to be a *one-click* (or close to it) operation, ideally automated with a tool such as Ansible, so that students do not need deep system-administration skills just to get started;

this is the same need that drives the executable setups in Levels 2–4. Second, the system has to be populated with **use-case-specific synthetic data** (for example atrial fibrillation, stroke/CVA, or mental-health cohorts) so that each learning objective has realistic data to work with. These two requirements are, in effect, the bridge between the education vision and the reference-implementation work: the same scripts that make OpenMRS reproducible for a developer also make it approachable for a student.

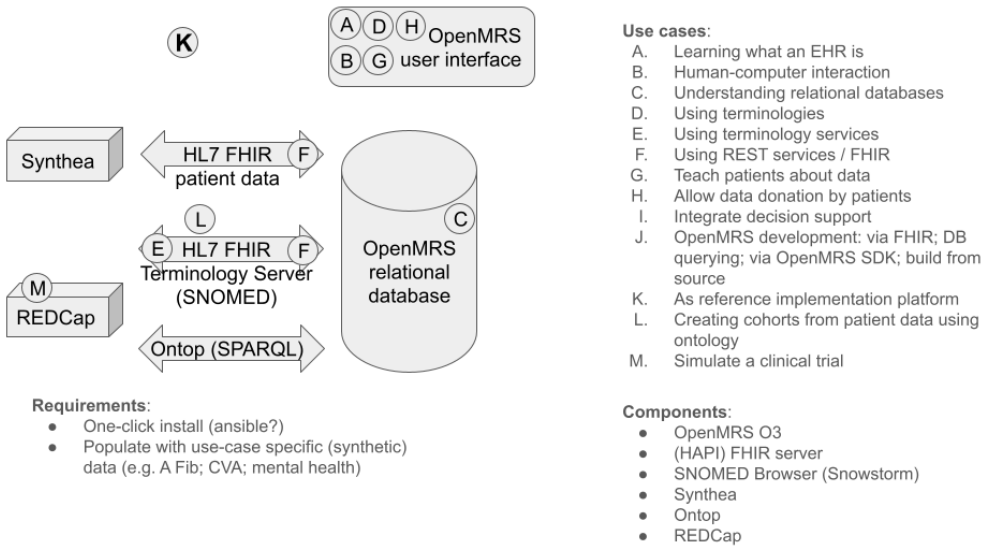

Figure 1: A long-term vision for using an open-source EHR (OpenMRS) as a platform for medical informatics education and research. Centre: the components of the platform. Right: learning objectives / use cases (A–M), positioned on the components that exercise them.

Discussion

Across the four levels, a pattern emerged. The "use it" levels (1 and 2) are reachable today: OpenMRS can already serve as a data source and as a locally queryable database, which is enough to prototype demos and to teach the core skills. The "change it" levels (3 and 4) are where the work remains. Adding a module and building from source are both *reproducible* now, with a scripted VM for each, but neither is yet *reliable*. The blockers we hit (an interactive SDK setup at Level 3; a `master`-vs-`2.8.5` build discrepancy and a non-working legacy UI at Level 4) are concrete and hand-off-ready rather than vague.

A second observation is that the developer-facing and education-facing goals reinforce each other. A one-click, reproducible install with use-case-specific synthetic data is simultaneously what a CDSS researcher needs to stand up a reference implementation and what a teacher needs to run a course. Investing in the install/build automation therefore pays off twice.

Finally, the genomic-CDS use case is a reminder of *why* the harder levels matter. Demonstrating standards-based, EHR-integrated decision support (CDS Hooks, SMART-on-FHIR, an up-to-date FHIR module) is precisely the kind of thing that cannot be shown with a specification alone, and it is what Levels 3 and 4 are meant to make possible.

Future work

- **Level 1:** contribute to the OpenMRS FHIR module (OpenMRS community, n.d.-c) to bring its FHIR version up to date, enabling modern CDS Hooks / SMART-on-FHIR

integrations.

- **Level 3:** drive the OpenMRS SDK non-interactively so that scaffolding and running a minimal module becomes a single repeatable command, then add the canonical "clickable URL on the patient summary" module.
- **Level 4:** resolve the `master` build error, restore the legacy UI, and make property/message changes visible, turning the source-build sandbox into a documented, reliable recipe.
- **Education platform:** deliver a one-click (e.g. Ansible-based) install that provisions OpenMRS together with the supporting components (Synthea, a FHIR server, a SNOMED terminology server, Ontop, REDCap), pre-populated with use-case-specific synthetic data.
- **Genomic CDS reference implementation:** building on the above, demonstrate an end-to-end pharmacogenomics CDSS (e.g. thiopurine or warfarin dosing) integrated with OpenMRS as a worked reference implementation.

Acknowledgements

We thank the OpenMRS community and the organisers and participants of the SWAT4HCLS 2026 Biohackathon at AmsterdamUMC.

AI usage acknowledgement

A large language model (Anthropic's Claude, via the Claude Code assistant) was used to help consolidate this intermediate report. Specifically, it was used to merge the authors' existing notes, the Biohackathon help-sheet descriptions of the four levels, a separate genomic clinical-decision-support concept document, and the education diagram into a single draft, and to assist with copy-editing and reference formatting. The AI did not generate any of the underlying results, data, code, or design decisions; all technical content reflects work performed by the authors. The authors reviewed, corrected, and approved the entire text and take full responsibility for its content.

References

Free Software Foundation. (n.d.). *What is Free Software?* https://www.gnu.org/philosophy/free-sw.en.html.

Herman, E. (n.d.). *openmrs-sandbox: building OpenMRS core from source*. https://gitlab.com/ericherman/openmrs-sandbox.

Kawamoto, K., Houlihan, C. A., Balas, E. A., & Lobach, D. F. (2005). Improving clinical practice using clinical decision support systems: a systematic review of trials to identify features critical to success. *BMJ*, *330*(7494), 765. https://doi.org/10.1136/bmj.38398.500764.8F

Medlock, S., Ploegmakers, K. J., Cornet, R., & Pang, K. W. (2023). Use of an open-source electronic health record to establish a "virtual hospital": A tale of two curricula. *International Journal of Medical Informatics*, *169*, 104907. https://doi.org/10.1016/j.ijmedinf.2022.104907

OpenMRS community. (n.d.-a). *Getting Started as a Developer*. https://openmrs.atlassian.net/wiki/spaces/docs/pages/25477022/Getting+Started+as+a+Developer.

OpenMRS community. (n.d.-b). *OpenMRS Reference Application distribution*. https://github.com/openmrs/openmrs-distro-referenceapplication.

OpenMRS community. (n.d.-c). *openmrs-module-fhir2*. https://github.com/openmrs/openmrs-module-fhir2.

OpenMRS community. (n.d.-d). *Set Up an Instance of O3*. https://openmrs.atlassian.net/wiki/spaces/docs/pages/150930190/Set+Up+an+Instance+of+O3.

OpenMRS Education. (n.d.). *openmrs-sdk-sandbox*. https://gitlab.com/openmrs_education/openmrs-sdk-sandbox.

Shen, Y., Yu, J., Zhou, J., & Hu, G. (2025). Twenty-Five Years of Evolution and Hurdles in Electronic Health Records and Interoperability in Medical Research: Comprehensive Review. *Journal of Medical Internet Research*, *27*, e59024. https://doi.org/10.2196/59024

Sittig, D. F., Wright, A., Osheroff, J. A., Middleton, B., Teich, J. M., Ash, J. S., Campbell, E., & Bates, D. W. (2008). Grand challenges in clinical decision support. *Journal of Biomedical Informatics*, *41*(2), 387–392. https://doi.org/10.1016/j.jbi.2007.09.003

Author contributions

Andra Waagmeester: Writing – original draft Stephanie "Ace" Medlock: Writing – original draft Eric Herman: Conceptualization Claude Nanjo: Writing – original draft Chang Sun: Conceptualization Ronald Cornet: Writing – original draft